TravelGo: A Cloud-Based Real-Time Travel Booking Platform Using AWS

Project Description

TravelGo is a web application designed to simplify travel booking by providing buses, trains, flights, and hotel reservations in one platform. Instead of using different websites for each service, users can search and book travel options from a single interface.

The backend of the application is developed using Flask in Python and deployed on an Amazon EC2 instance. User information and booking details are stored in AWS DynamoDB, which allows the system to manage data efficiently.

Users can register, log in, search for travel options, and complete bookings through the platform. After a booking is confirmed or cancelled, the system sends an email notification using AWS SNS to inform the user about the status of the reservation.

The platform also includes features such as bus seat selection, hotel filtering options like luxury or budget, and a dashboard where users can view or cancel their bookings. By combining Flask with AWS cloud services, TravelGo provides a simple and efficient travel booking experience.

Scenario 1: Multi-Service Travel Booking

TravelGo allows users to search and book buses, trains, flights, and hotels from one platform. For example, a user traveling from Hyderabad to Bangalore can log in, check available options, and complete the booking. Since the system is hosted on AWS EC2, it can handle multiple users at the same time.

Scenario 2: Instant Booking Notifications

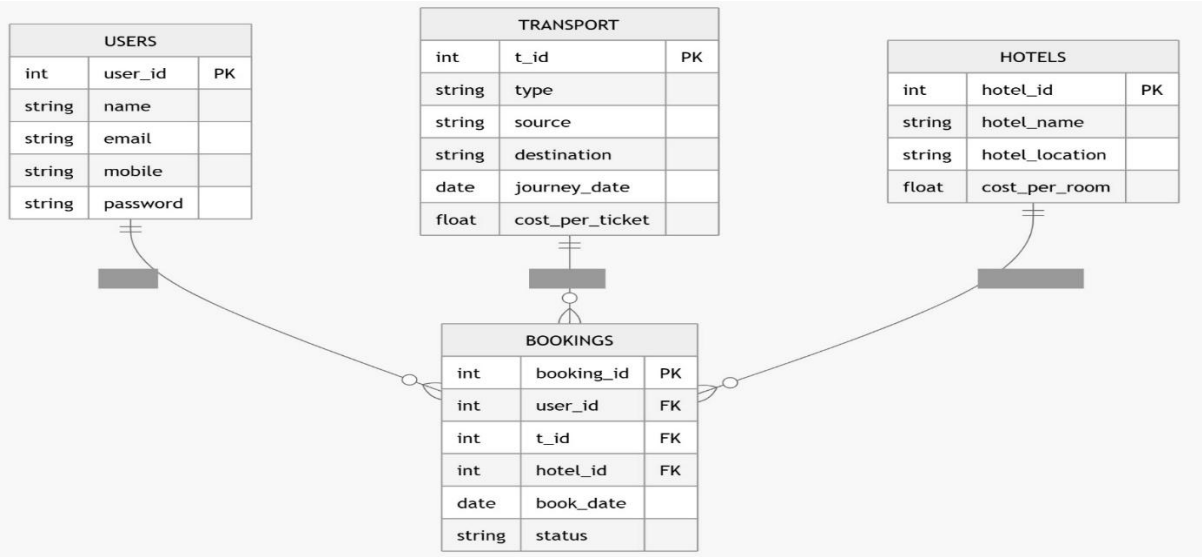
After a booking is completed, the system sends a confirmation email using AWS SNS. For instance, if a user books a hotel in Chennai, the system automatically sends the booking details to the user’s email after the information is saved in DynamoDB.

Scenario 3: User Dashboard and Booking History

The platform provides a dashboard where users can view their past and upcoming bookings. Details such as travel type, date, and price are displayed. Users can also cancel bookings if required. The data is retrieved from DynamoDB and displayed through the web interface.

AWS ARCHITECTURE

ENTITY RELATIONSHIP (ER) DIAGRAM:



Pre-requisites:

1. **.AWS Account setup:** AWS ACCOUNT SETUP
2. **Uderstanding IAM :** IAM Overview
3. **Amazon EC2 Basics:** EC2 Tutorial
4. **DynamoDB Basics:** DynamoDB Introduction
5. **SNS Overview:** SNS Documentation
6. **Git Version Control:** Git Documentation

Project Workflow

1. AWS Account Setup and Login

- **Activity 1.1:** Set up an AWS account if not already created.
- **Activity 1.2:** Log in to the AWS Management Console.

2. DynamoDB Database Creation and Setup

- **Activity 2.1:** Create a DynamoDB table.
- **Activity 2.2:** Configure attributes for **Users**, **Transport, Hotels** and **Bookings** tables.

3. SNS Notification Setup

- **Activity 3.1:** Create an SNS topic named **BookingConfirmation** for sending notifications.
- **Activity 3.2:** Subscribe an email to the topic to receive booking and cancellation alerts.

4. Backend Development and Application Setup

- **Activity 4.1:** Develop the backend using Flask.
- **Activity 4.2:** Integrate AWS services using **boto3**.

5. IAM Role Setup

- **Activity 5.1:** Create an IAM role.
- **Activity 5.2:** Attach required policies.

6. EC2 Instance Setup

- **Activity 6.1:** Launch an EC2 instance to host the Flask application.
- **Activity 6.2:** Configure security groups for **HTTP** and **SSH** access.

7. Deployment on EC2

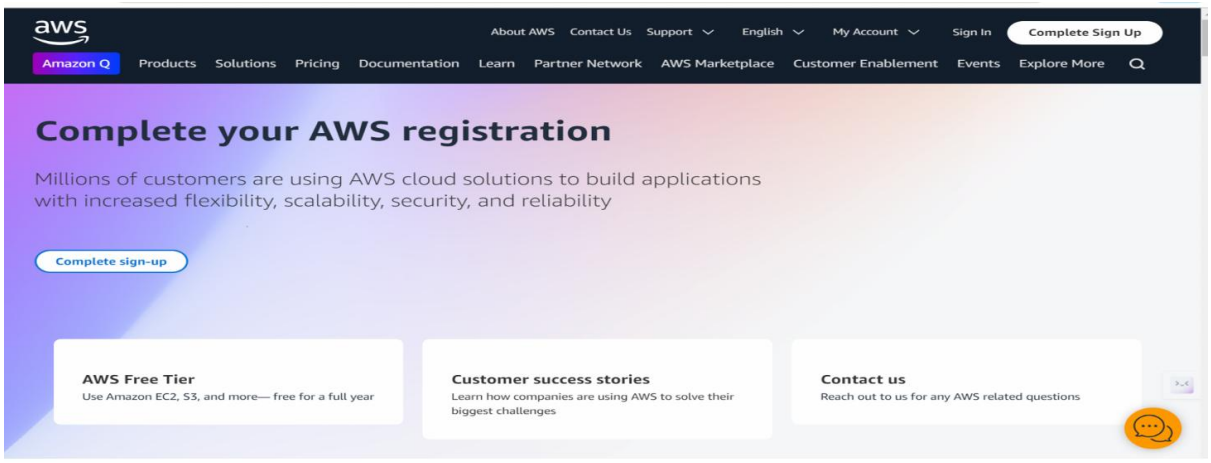
- **Activity 7.1:** Upload Flask files to the EC2 instance.
- **Activity 7.2:** Run the Flask application.

8. Testing and Deployment

- **Activity 8.1:** Conduct functional testing to verify user registration, login, booking requests, and notification delivery.

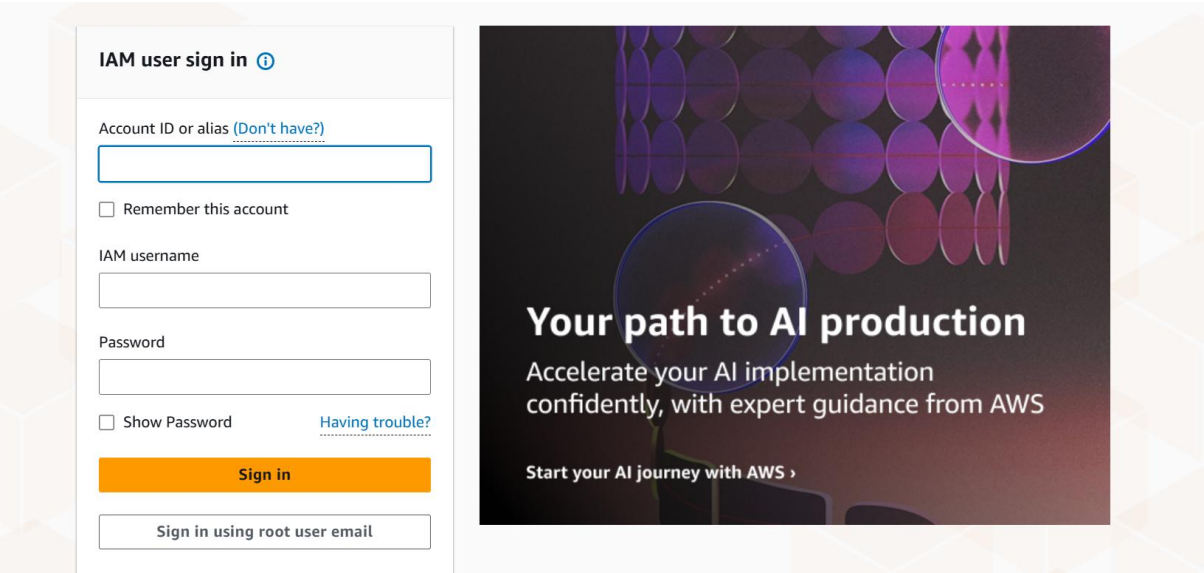
Milestone 1: AWS Account Setup and Login

Activity 1.1: Set up an AWS account if not already done



Sign up for an AWS account and configure billing settings.

Activity 1.2: Log in to the AWS Management Console

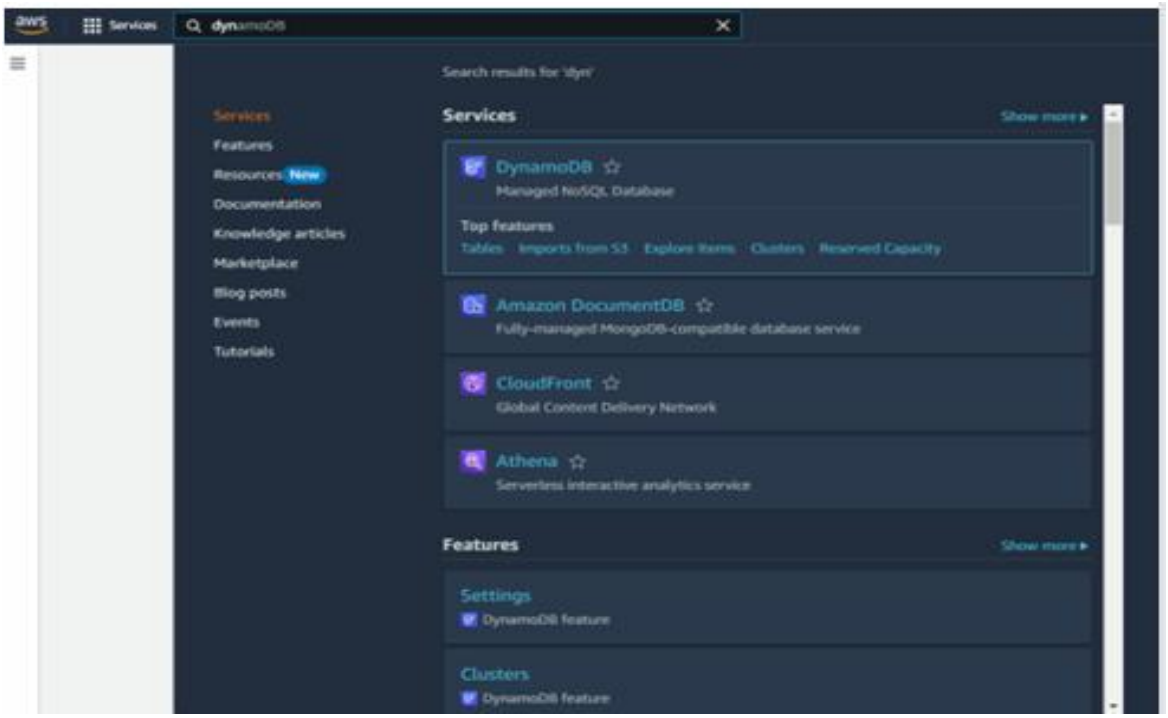


After setting up your account, log in to the AWS Management Console.

Milestone 2: DynamoDB Database Creation and Setup

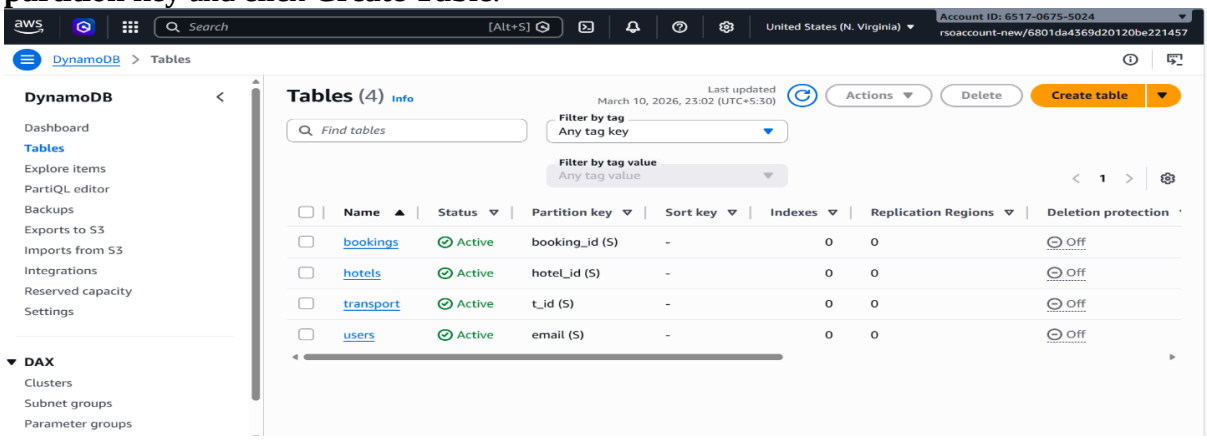
Activity 2.1: Navigate to DynamoDB

In the AWS Console, navigate to DynamoDB and click on **Create Table**.



Activity 2.2: Create DynamoDB tables for storing registration details and booking records

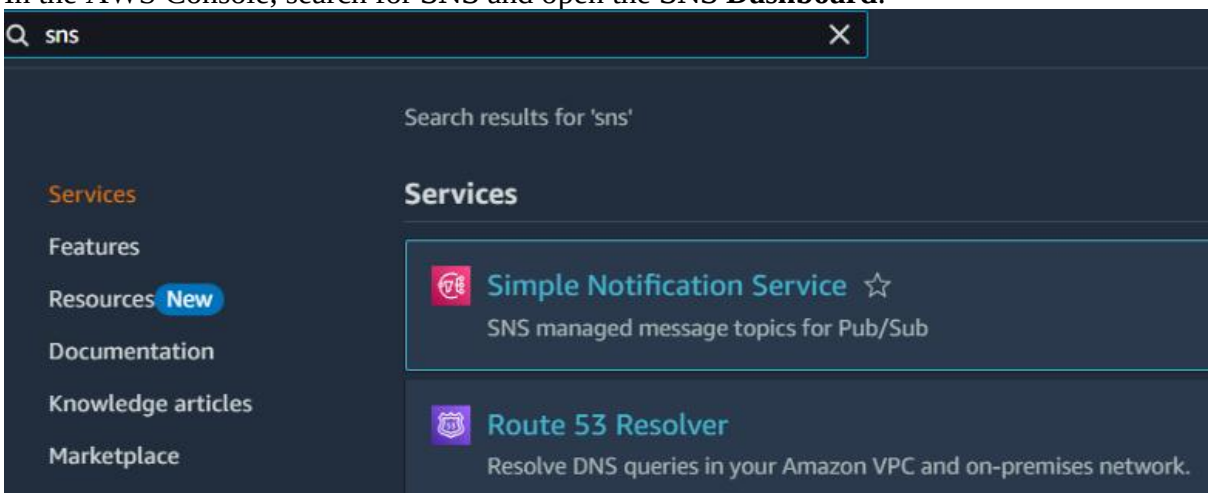
- Create a **users** table for storing registered user details with partition key **Email (String)** and click **Create Table**.
- Create a **transport** table for storing details of transport types with partition key as **t_id** (**Int**) and click **Create Table**.
- Create a **Hotels** table for storing details of hotels like location,name with partition key **hotel_id**(**int**) and click **Create Table**.
- Create a **Bookings** table for storing booking records with **booking_id as the partition key** and click **Create Table**.



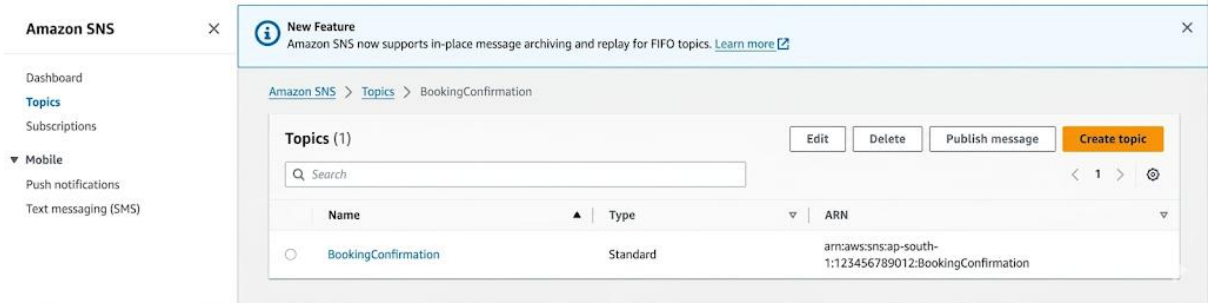
Milestone 3: SNS Notification Setup

Activity 3.1: Create an SNS Topic named BookingConfirmation for sending notifications

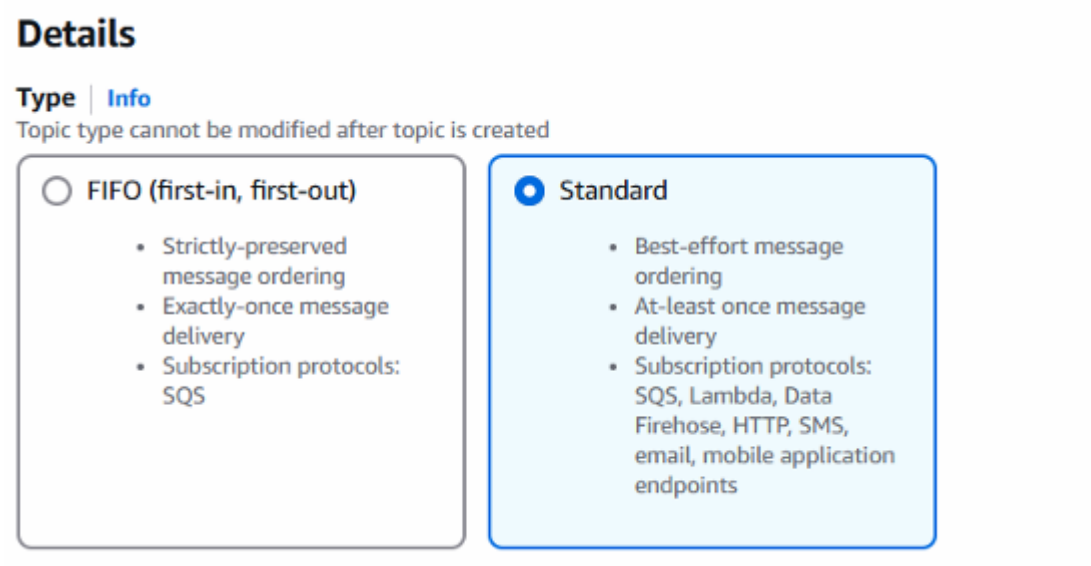
- In the AWS Console, search for **SNS** and open the **SNS Dashboard**.



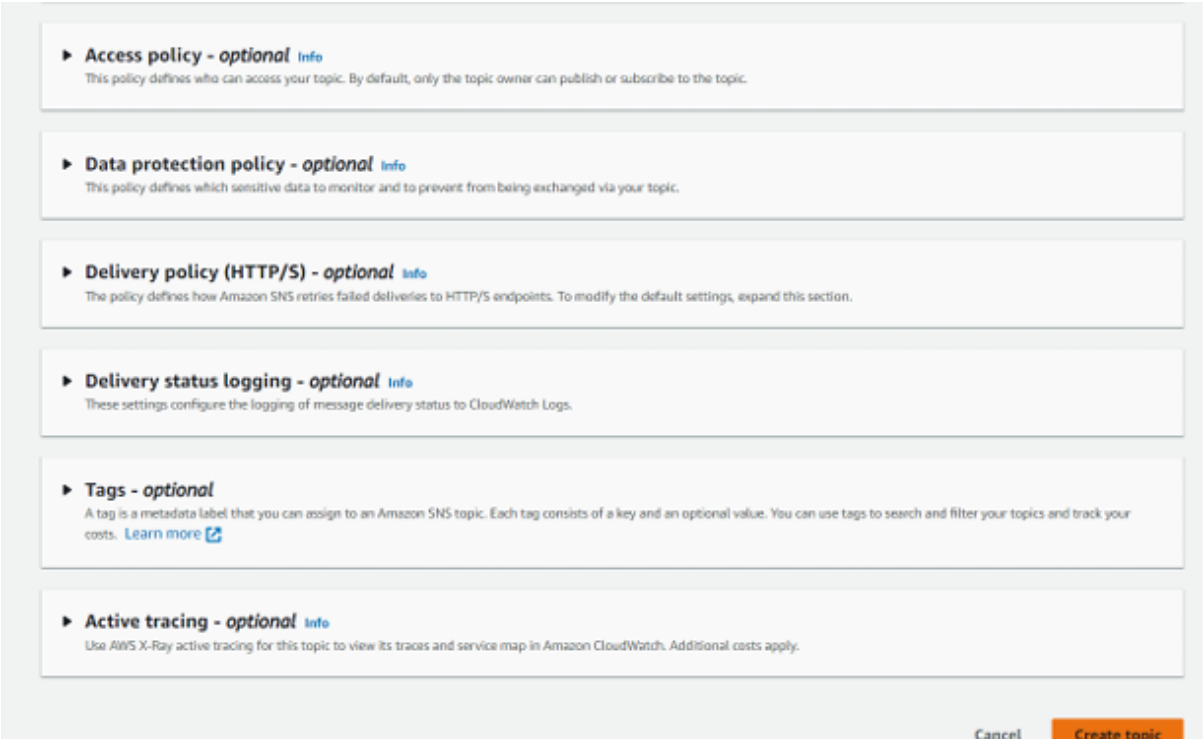
- Click **Create Topic** and choose a name for the topic.



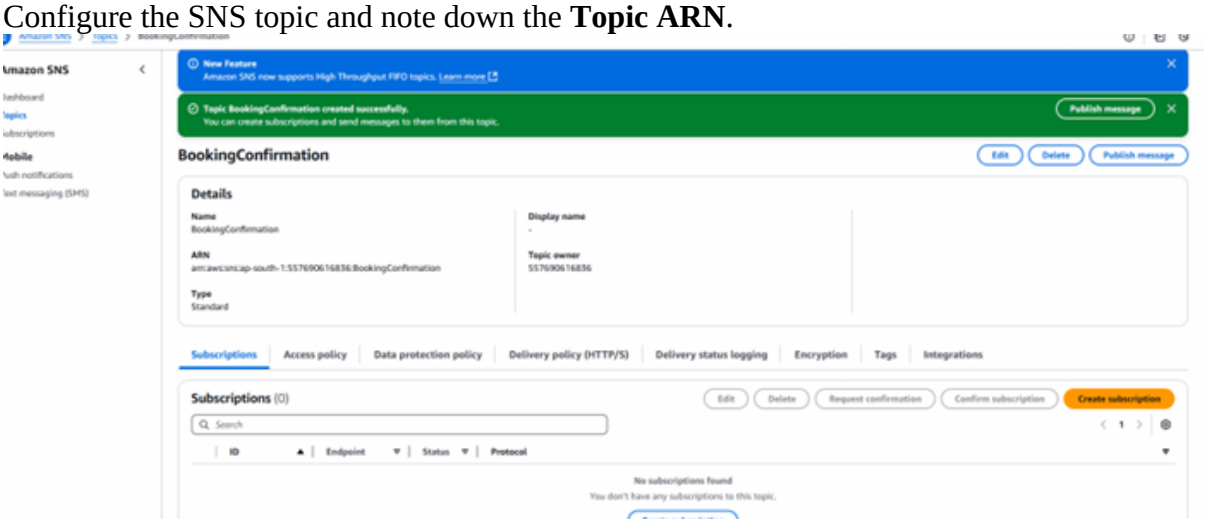
- Choose **Standard type** for general notification use cases.



- Click **Create Topic**.

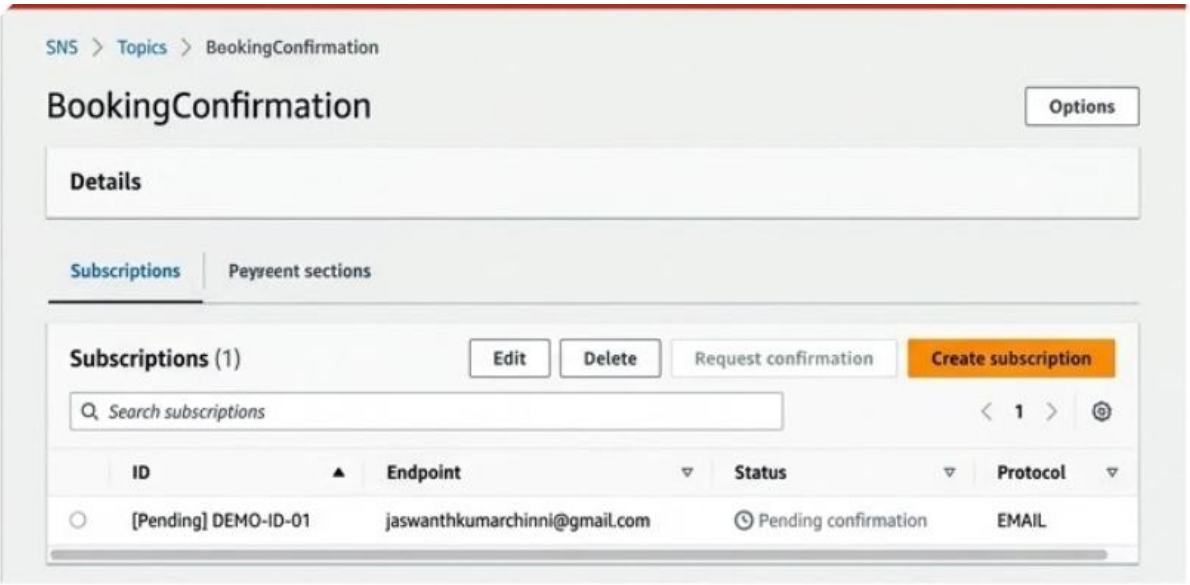


- Configure the SNS topic and note down the **Topic ARN**.

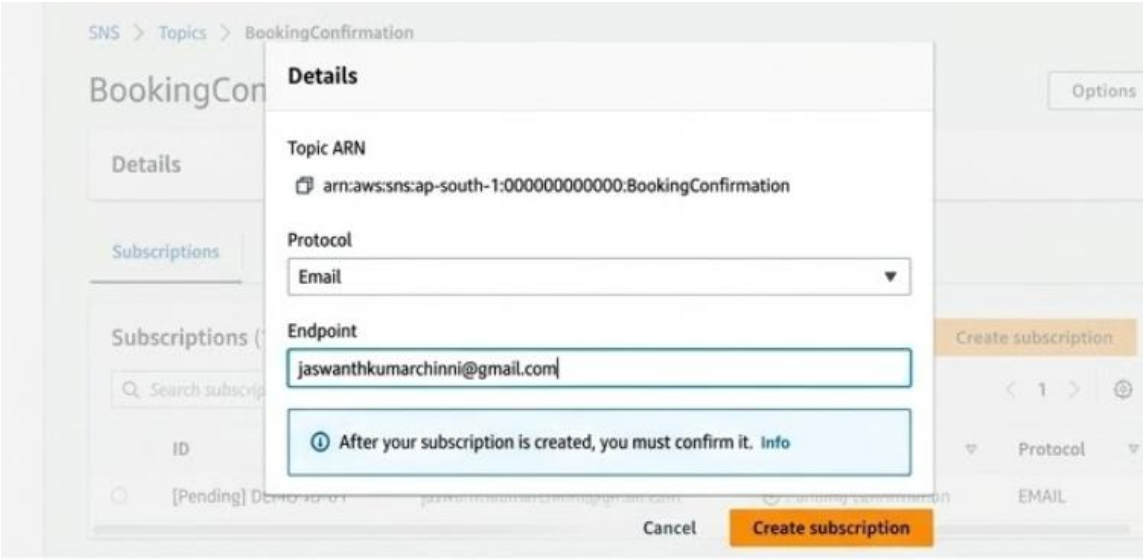


Activity 3.2: Subscribe email to the topic to receive booking and cancellation alerts

- Subscribe users (or admin staff) to the topic via **Email**.



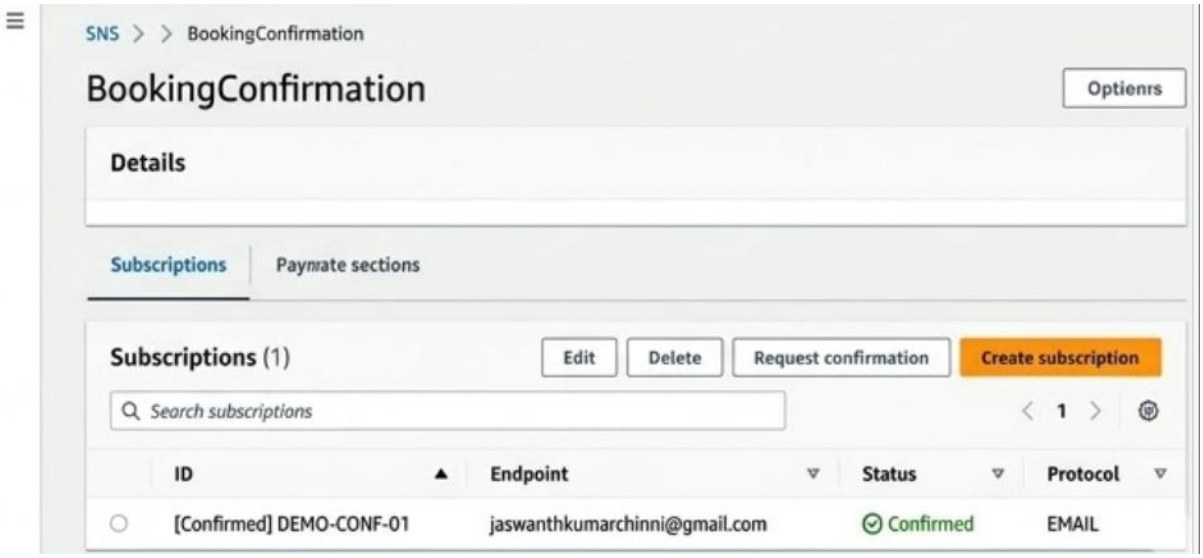
- When a booking request is made, notifications will be sent to the subscribed emails.



After receiving the subscription request email:

- Navigate to the subscribed email account.

- Click **Confirm Subscription** in the **AWS Notification – Subscription Confirmation** email.

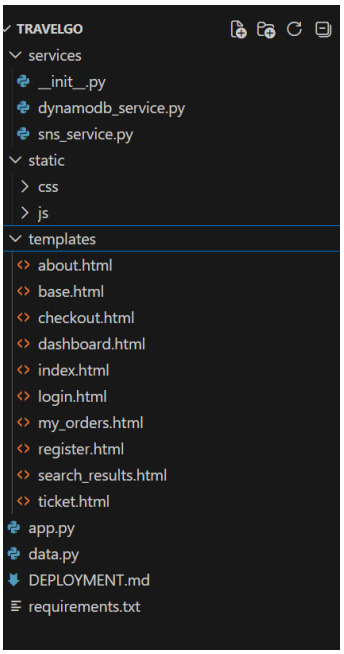


Successfully complete the SNS email subscription setup and store the **ARN link**.

Milestone 4: Backend Development and Application Setup

Activity 4.1: Develop the Backend Using Flask

File Explorer Structure



Description: Create a templates folder containing HTML files for each major feature—bus, train, flight, hotels, seat selection, registration, login, payment, and dashboard—and manage backend logic in app.py and data.py.

Description of the Code

Flask App Initialization

Import Libraries

```python
app.py > ...
import os
import uuid
import datetime
from flask import Flask, render_template, request, redirect, flash, session, jsonify
from services.dynamodb_service import (
    create_user, get_user, create_booking, get_user_bookings,
    cancel_booking, get_booking
)
from services.sns_service import send_notification
from data import TRANSPORT_DATA, HOTEL_DATA

```

Import necessary Flask modules for web routing, **Boto3** for AWS DynamoDB access, **UUID** for unique IDs, and custom transport and hotel data from **data**.

Initialize Flask App

```python
app = Flask(__name__)
```

Initialize the Flask application by creating an instance of the Flask class with the current module name.

DynamoDB Connection

```python
# Updated default region to N. Virginia (us-east-1)
REGION      = os.getenv('AWS_DEFAULT_REGION', 'us-east-1')
USE_MOCK_DB = os.getenv('USE_MOCK_DB', 'false').lower() == 'true'

# — In-memory store (used automatically if DynamoDB is unreachable) —
_mem_users    = {}   # email -> user_dict
_mem_bookings = {}   # booking_id -> booking_dict
_using_memory = False

def _check_dynamo():
    """Try a cheap call to see if DynamoDB is reachable. Returns True if OK."""
    try:
        # This will now target us-east-1 by default
        boto3.client('dynamodb', region_name=REGION).list_tables(Limit=1)
        return True
    except Exception as e:
        print(f"[DB] DynamoDB unreachable in {REGION}, using in-memory store: {e}")
        return False
# Decide at startup (or first use)
_dynamo_ok = None
```

Use **boto3** to connect to DynamoDB for handling **travel-users** and **bookings** database operations and specify the **region_name** where DynamoDB tables are created.

SNS Connection

```python
def get_sns_client():
    return boto3.client('sns', region_name=REGION)
```

Configure **SNS** to send notifications when a booking request is submitted.

Routes for Web Pages

Register User

```python
@app.route('/')
def home():
    return render_template('index.html', logged_in='user' in session)
```

```python
@app.route("/register", methods=["GET", "POST"])
def register():
    if 'user_email' in session:
        return redirect("/dashboard")

    if request.method == "POST":
        name     = request.form.get("name", "").strip()
        email    = request.form.get("email", "").strip().lower()
        mobile   = request.form.get("mobile", "").strip()
        password = request.form.get("password", "")
        confirm  = request.form.get("confirm_password", "")

        if not all([name, email, mobile, password]):
            flash("All fields are required.", "error")
            return render("register.html")
```

Collect registration data and store user details in the database.

Login Route (GET/POST)

```python
@app.route("/login", methods=["GET", "POST"])
def login():
    if 'user_email' in session:
        return redirect("/dashboard")

    if request.method == "POST":
        email    = request.form.get("email", "").strip().lower()
        password = request.form.get("password", "")
        try:
            user = get_user(email)
            if user and user.get("password") == password:
                session['user_email'] = email
                session['user_name']  = user.get("name", "Traveller")
                session['user_id']    = user.get("user_id", "")
                flash(f"Welcome back, {user.get('name', 'Traveller')}! Ready to explore?", "success")
                return redirect("/dashboard")
            else:
                flash("Invalid email or password.", "error")
        except Exception as e:
            flash(f"Login error: {str(e)}", "error")

    return render("login.html")
```

Verify user credentials, increment login count, and redirect to the dashboard on success.

Dashboard Route

```python
@app.route("/dashboard")
def dashboard():
    if 'user_email' not in session:
        flash("Please login to access your dashboard.", "error")
        return redirect("/login")
    try:
        bookings = get_user_bookings(session['user_email'])
    except Exception as e:
        print(f"[DB] Could not fetch bookings: {e}")
        bookings = []
    return render("dashboard.html",
                  user_email=session.get('user_email'),
                  bookings=bookings)
```

Display the user's booking data from DynamoDB after verifying the session and fetching user details using their email.

Transport Routes

```python
@app.route('/bus', methods=['GET', 'POST'])
def bus():
    if request.method == 'POST':
        s, d, dt = request.form['source'], request.form['destination'], request.form['date']
        buses = [b for b in transport_data['bus'] if b['source'] == s and b['destination'] == d]
        return render_template('bus.html', buses=buses, source=s, destination=d, date=dt)
    return render_template('bus.html', buses=None)

@app.route('/train', methods=['GET', 'POST'])
def train():
    if request.method == 'POST':
        s, d, dt = request.form['source'], request.form['destination'], request.form['date']
        trains = [t for t in transport_data['train'] if t['source'] == s and t['destination'] == d]
        return render_template('train.html', trains=trains, source=s, destination=d, date=dt)
    return render_template('train.html', trains=None)

@app.route('/flight', methods=['GET', 'POST'])
def flight():
    if request.method == 'POST':
        s, d, dt = request.form['source'], request.form['destination'], request.form['date']
        flights = [f for f in transport_data['flight'] if f['source'] == s and f['destination'] == d]
        return render_template('flight.html', flights=flights, source=s, destination=d, date=dt)
    return render_template('flight.html', flights=None)

@app.route('/hotels', methods=['GET', 'POST'])
def hotels():
    if request.method == 'POST':
        city = request.form['city']
        hotels = [h for h in hotel_data if h['location'] == city]
        return render_template('hotels.html', hotels=hotels, city=city)
```

Define routes for **bus, train, flight, and hotel searches** using POST requests to filter results based on source, destination, date, or city.
Also include **seat selection routes** for bus bookings.

Booking Routes

```python
@app.route("/confirm_booking", methods=["POST"])
def confirm_booking():
    if 'user_email' not in session:
        flash("Session expired. Please login again.", "error")
        return redirect("/login")

    co          = session.get('checkout', {})
    mode        = co.get("mode", "flight")
    item        = co.get("item", {})
    travel_date = co.get("date", "")
    passengers  = int(co.get("passengers", 1))

    if not item:
        flash("Booking session expired. Please start again.", "error")
        return redirect("/search")

    email     = session['user_email']
    user_name = session.get('user_name', '')
    user_id   = session.get('user_id', '')

    booking_id    = "TRV" + str(uuid.uuid4())[:8].upper()
    cost_per_unit = float(item.get("cost", 0))
```

Save booking details in the session and redirect to the **payment page** if the user is logged in.

Payment Route

```python
@app.route('/payment', methods=['POST'])
def payment():
    if 'user' not in session or 'pending_booking' not in session:
        return redirect('/login')

    booking = session.pop('pending_booking')
    booking['payment_method'] = request.form['method']
    booking['payment_reference'] = request.form['reference']

    bookings_table.put_item(Item=booking)

    # SNS Booking Notification
    try:
        sns.publish(
            TopicArn=os.getenv('SNS_TOPIC_ARN'),
            Message=f"✅ Booking Confirmed!\n{booking['details']} on {booking['date']} for ₹{booking['price']}",
            Subject="TravelGo Booking Confirmation"
        )
    except Exception as e:
        print("SNS Error (booking):", e)

    return redirect('/dashboard')
```

Process payment details, store the booking in the database, and send an **SNS confirmation notification** before redirecting to the dashboard.

Remove Booking Route

```python
@app.route("/cancel_booking", methods=["POST"])
def cancel_booking_route():
    if 'user_email' not in session:
        flash("Please login to manage bookings.", "error")
        return redirect("/login")

    booking_id = request.form.get("booking_id", "")
    email      = session['user_email']
    user_name  = session.get('user_name', '')

    try:
        cancel_booking(booking_id, email)
        flash(f"Booking {booking_id} has been cancelled.", "success")
        try:
            send_notification(
                email,
                f"Hi {user_name},\n\nYour booking {booking_id} has been cancelled.\n\n— TravelGo Team
                f"TravelGo Booking Cancelled — {booking_id}"
            )
        except Exception as e:
            print(f"[SNS] Cancellation notification skipped: {e}")
    except Exception as e:
        flash(f"Cancellation failed: {str(e)}", "error")

    return redirect("/my_orders")

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=80, debug=True)
```

Delete a booking from the database and send a **cancellation notification via SNS**, then redirect to the dashboard.

Logout Route

```python
@app.route("/logout")
def logout():
    session.clear()
    flash("You've been logged out. Safe travels!", "success")
    return redirect("/")
```

Clear session data and reset **loggedIn** in localStorage before redirecting the user to the homepage.

Deployment Code

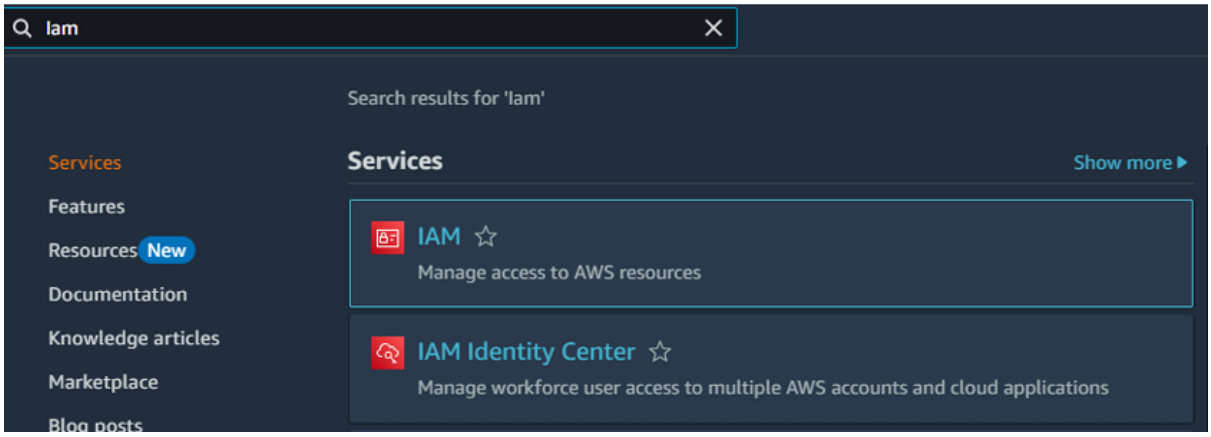
```python
if __name__ == "__main__":
    app.run(host='0.0.0.0', port=80, debug=True)
```

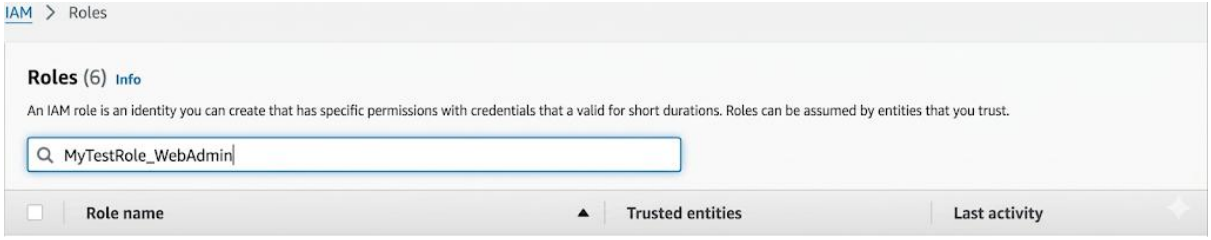
Start the Flask application on **port 80** with debug mode enabled.

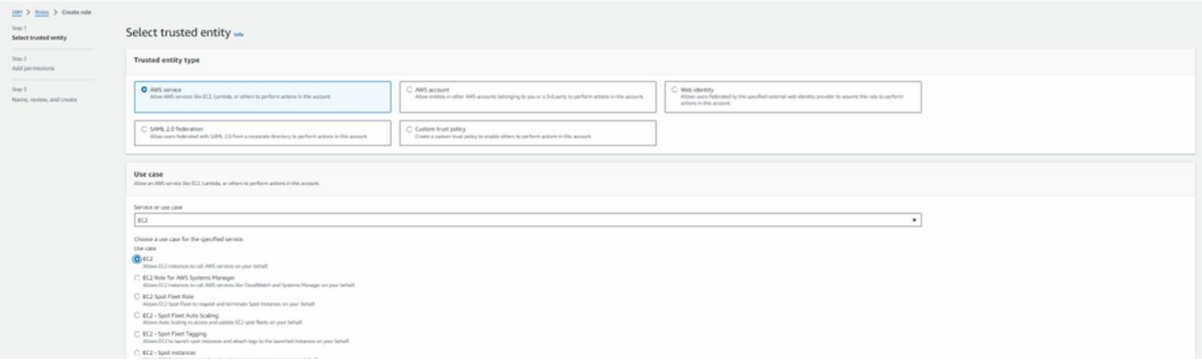
Milestone 5: IAM Role Setup

Activity 5.1: Create IAM Role

In the AWS Console, go to **IAM** and create a new **IAM Role for EC2** to interact with DynamoDB and SNS.



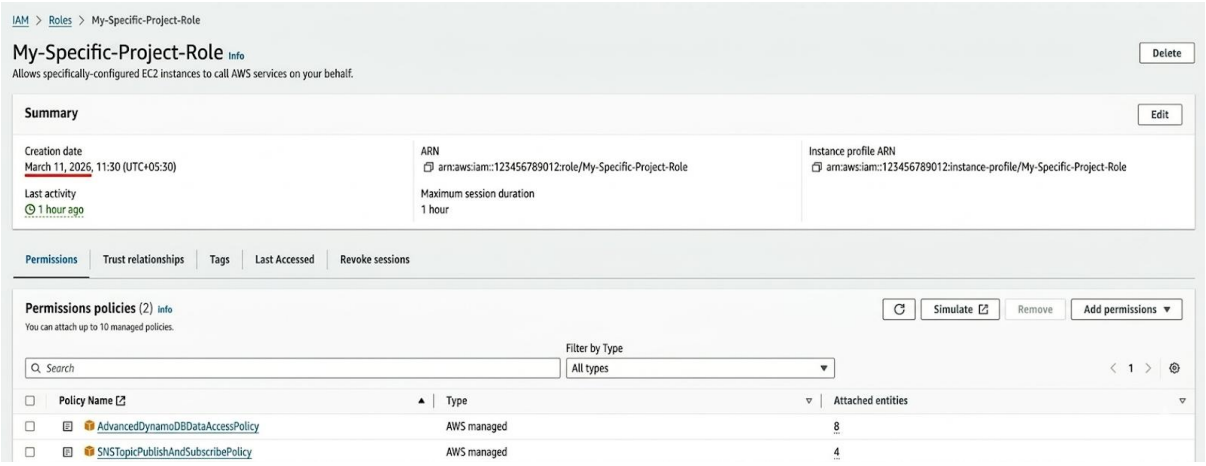


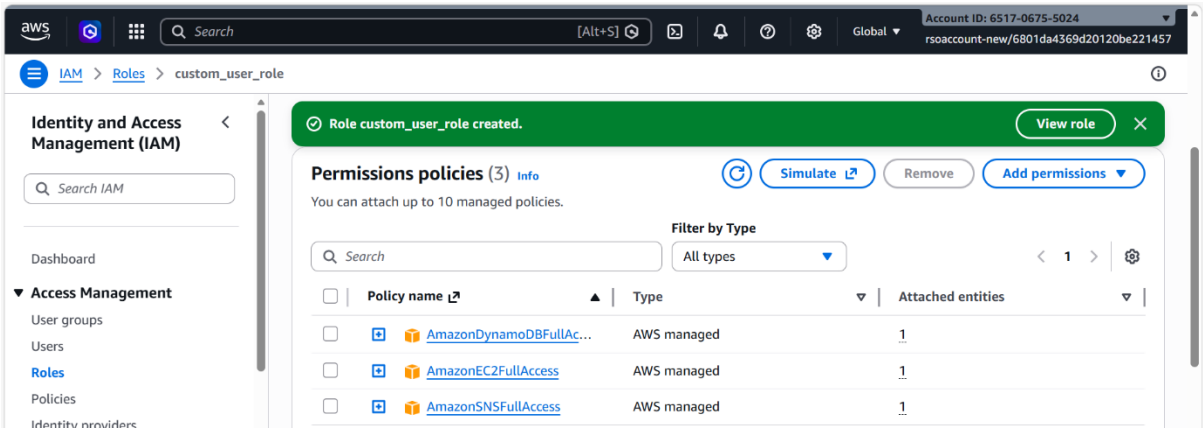


Activity 5.2: Attach Policies

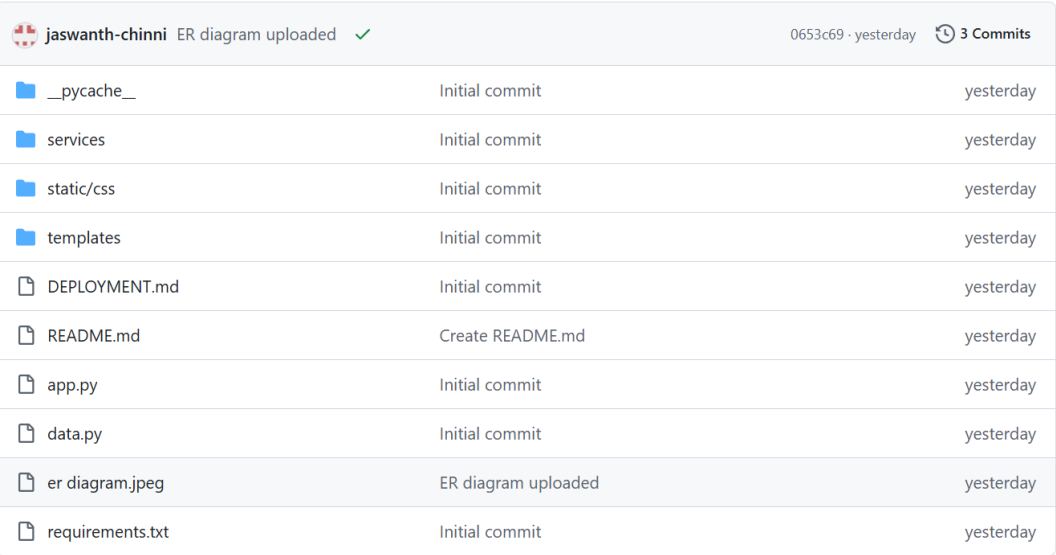
Attach the following policies:

- **AmazonDynamoDBFullAccess** – Allows EC2 to perform read/write operations on DynamoDB
- **AmazonSNSFullAccess** – Allows EC2 to send notifications through SNS



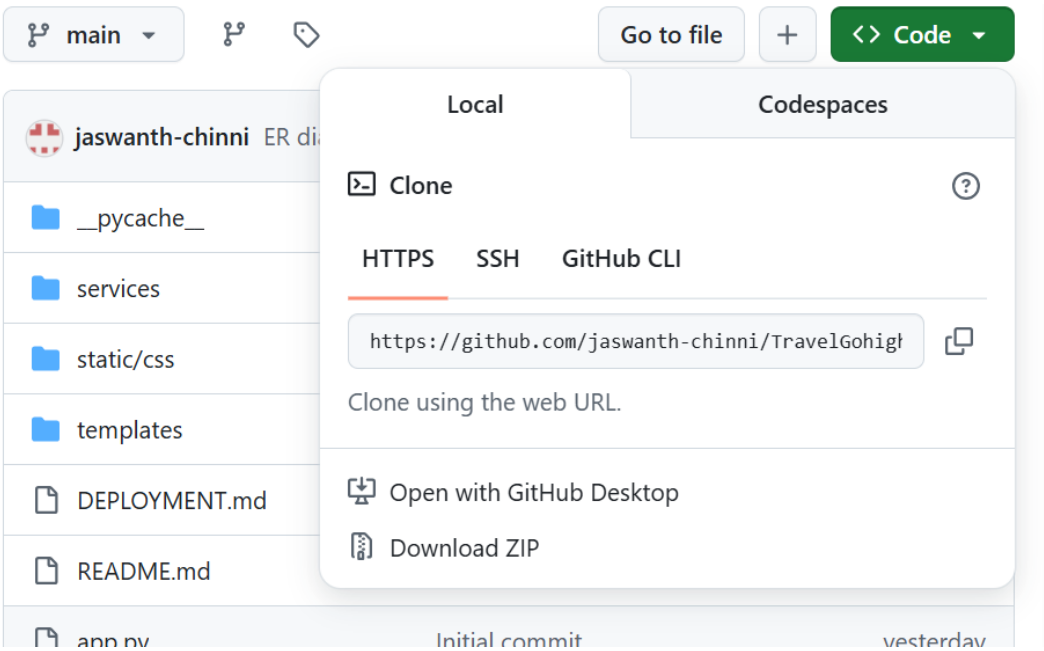


Milestone 6: EC2 Instance Setup



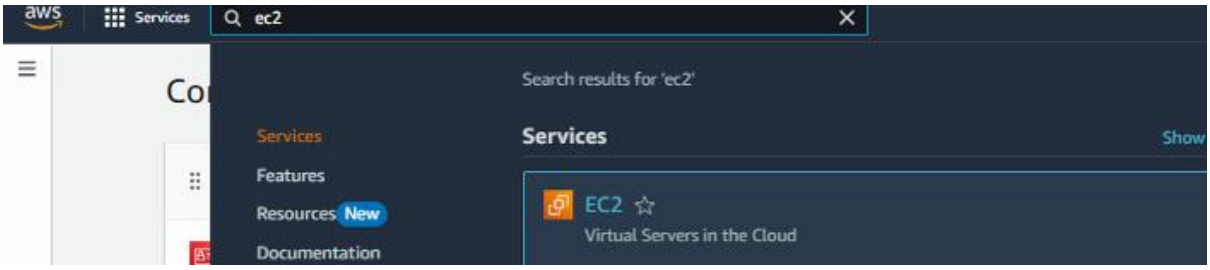
Note:

Upload your **Flask app and HTML files** to a **GitHub repository**.

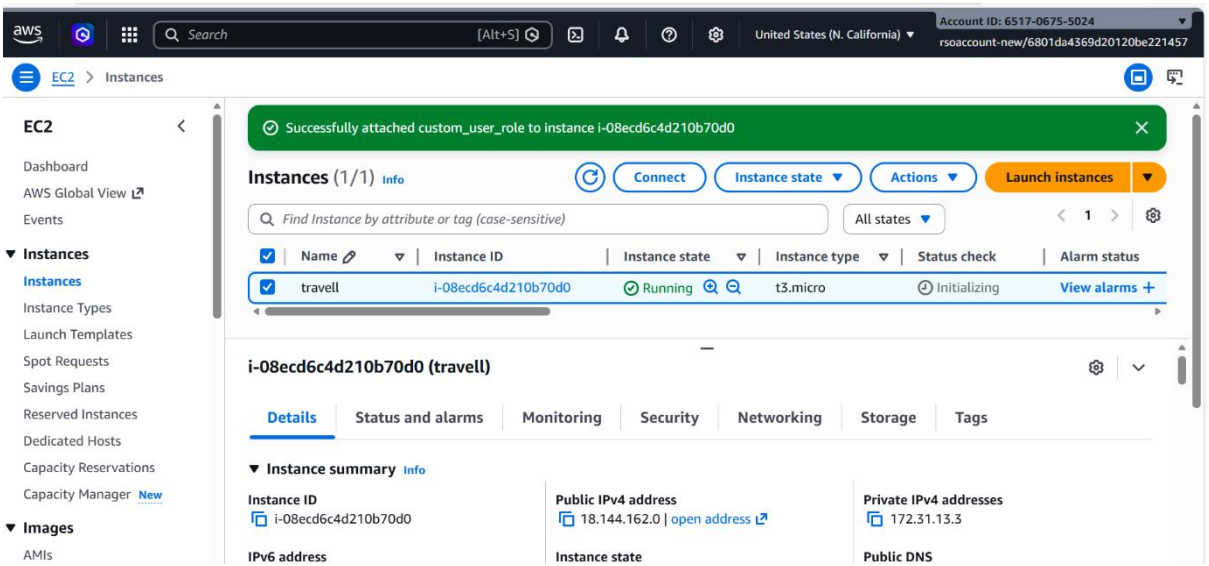


Activity 6.1: Launch an EC2 Instance

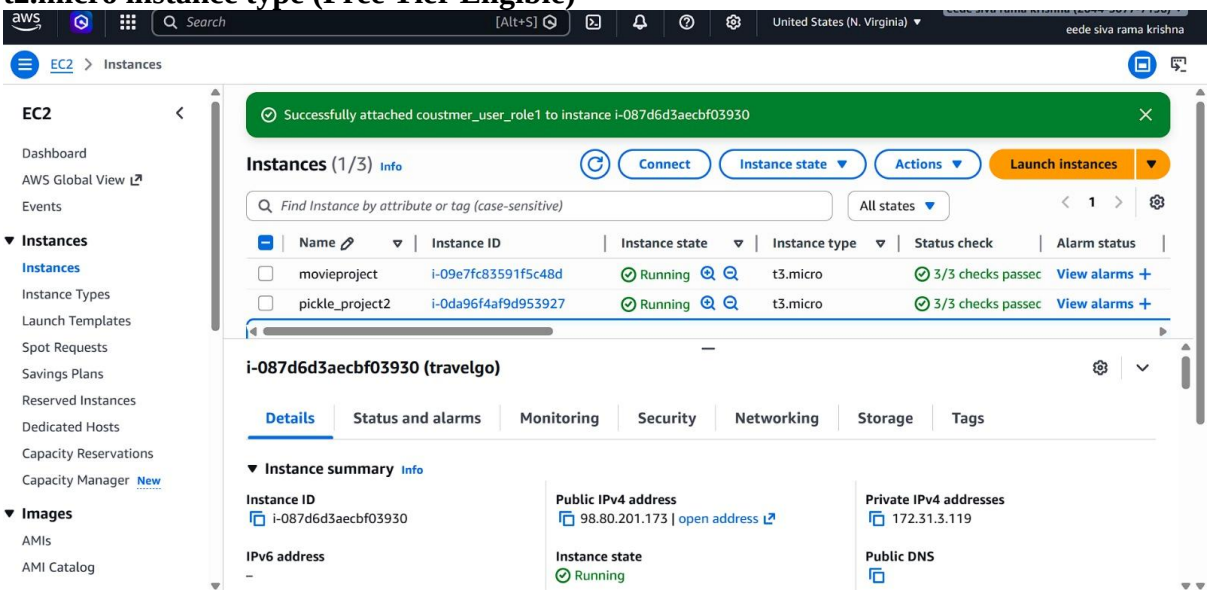
- Navigate to **EC2** in AWS Console



- Click **Launch Instance**



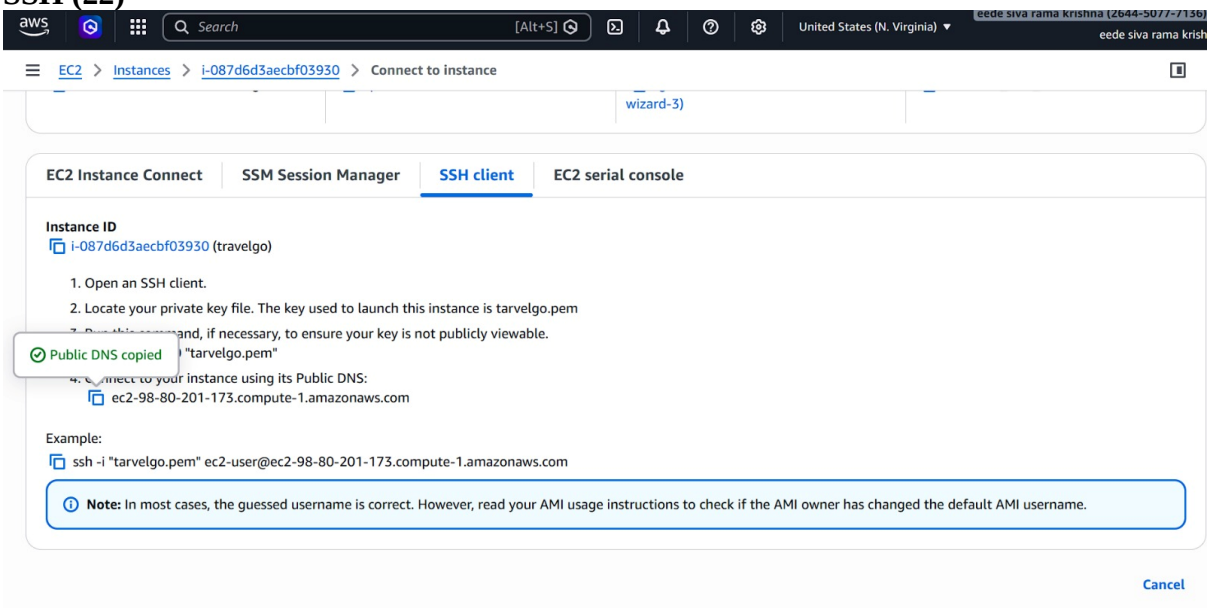
Choose:

- **Amazon Linux 2 or Ubuntu AMI**
- **t2.micro instance type (Free Tier Eligible)**
- 

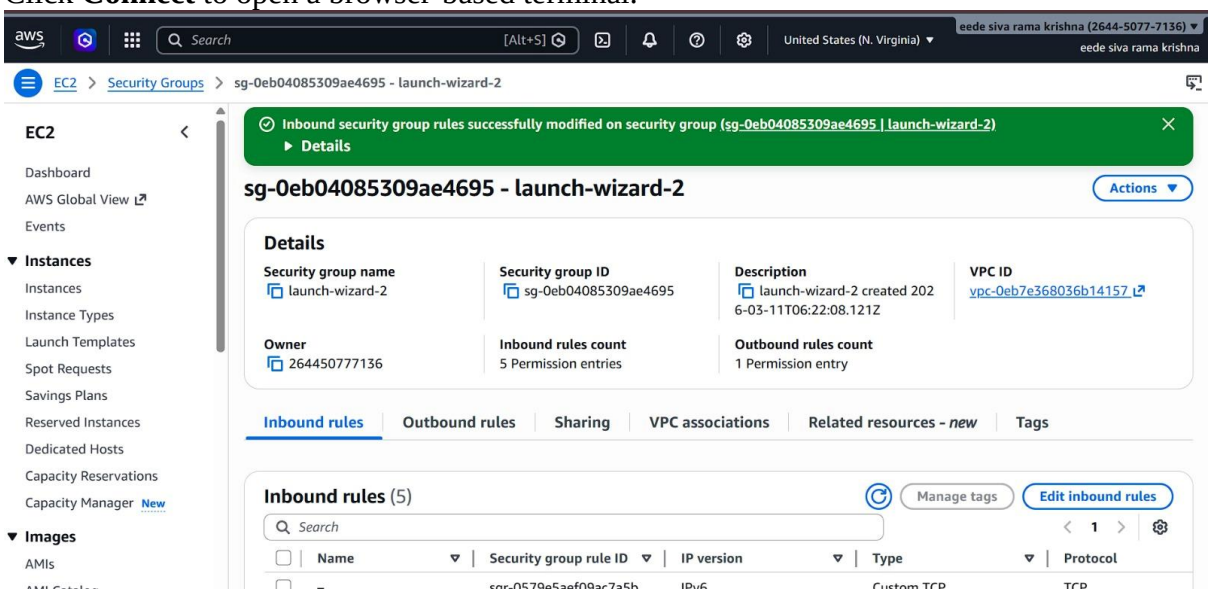
Create and download the **Key Pair** for server access.

Activity 6.2: Configure Security Groups

Allow the following ports:

- **HTTP (80)**
- **SSH (22)**
- 

To connect using **EC2 Instance Connect**:

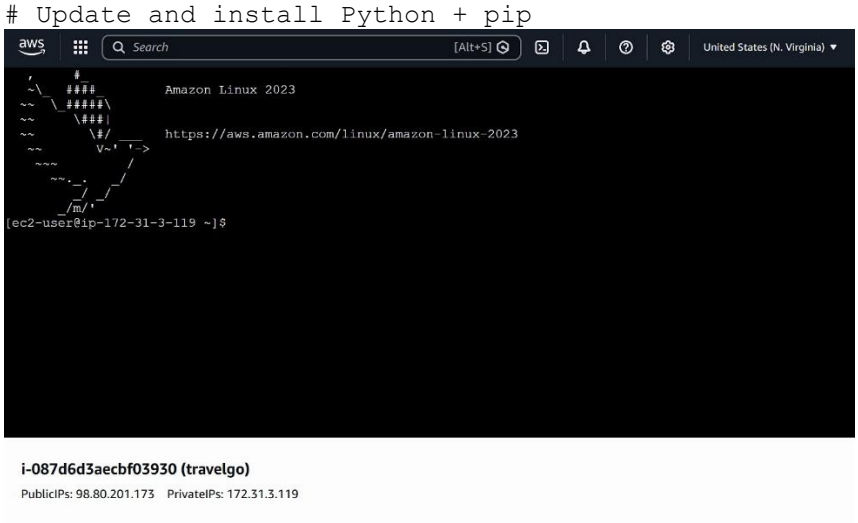
1. Ensure an **IAM role is attached** to the EC2 instance.
2. Select the instance.
3. Click **Connect**.
4. Choose **EC2 Instance Connect**.
5. Click **Connect** to open a browser-based terminal.
6. 

Now connect the EC2 instance with your project files.

Milestone 7: Deployment on EC2

Activity 7.1: Install Software on EC2

```
# Update and install Python + pip
```




```
sudo apt update
sudo apt install python3-pip -y
sudo apt install git -y
```

```
sudo apt install python3-venv -y

python3 -m venv venv
source venv/bin/activate
```

Activity 7.2: Clone Flask Project from GitHub

```
git clone https://github.com/jaswanth-chinni/TravelGohigh.git
```

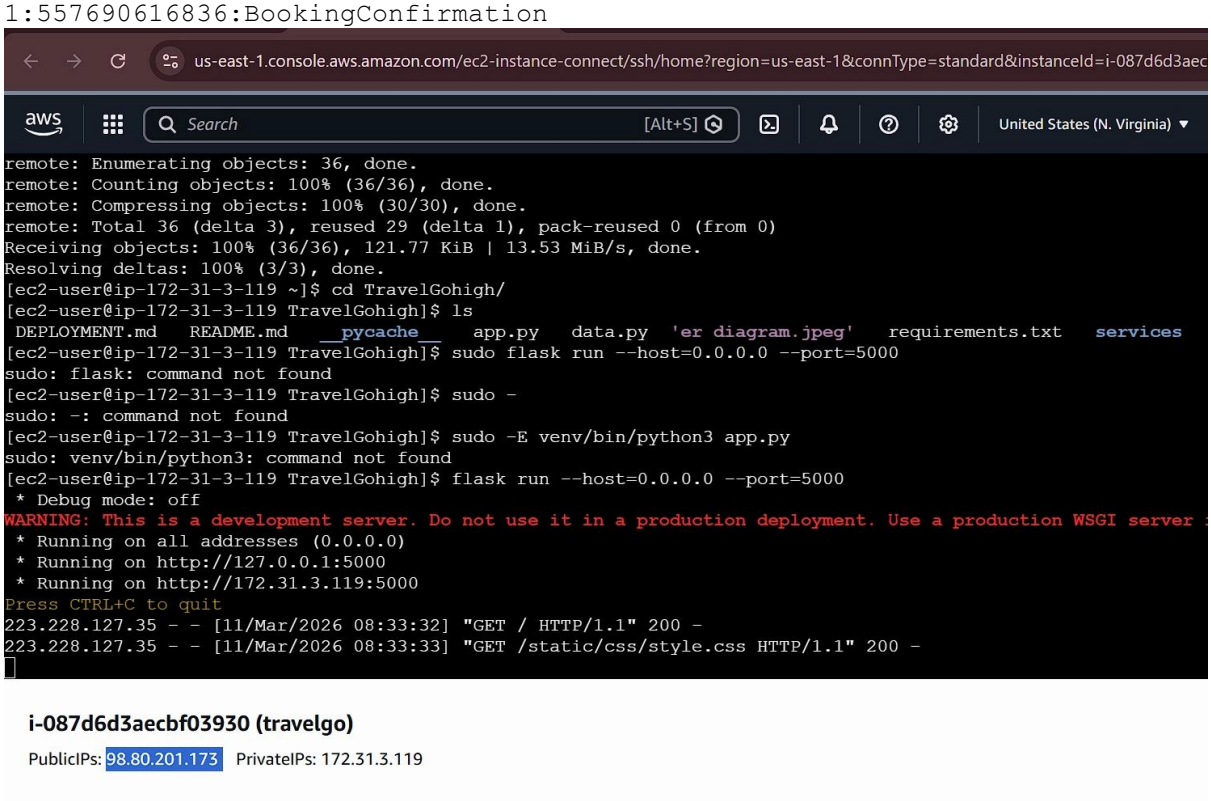
Navigate to project folder:

```
cd TravelGohigh
```

Run the Flask Application

Set the SNS Topic ARN:

```
export SNS_TOPIC_ARN=arn:aws:sns:ap-south-1:557690616836:BookingConfirmation
```



Install dependencies:

```
pip install flask boto3
```

```
Defaulting to user installation because normal site-packages is not writeable
Collecting boto3
  Downloading boto3-1.42.65-py3-none-any.whl (140 kB)
     |                                | 140 kB 19.8 MB/s
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in /usr/lib/python3.9/site-packages (from boto3) (0.10
Collecting botocore<1.43.0,>=1.42.65
  Downloading botocore-1.42.65-py3-none-any.whl (14.6 MB)
     |                                | 14.6 MB 51.8 MB/s
Collecting s3transfer<0.17.0,>=0.16.0
  Downloading s3transfer-0.16.0-py3-none-any.whl (86 kB)
     |                                | 86 kB 14.3 MB/s
Requirement already satisfied: urllib3<1.27,>=1.25.4 in /usr/lib/python3.9/site-packages (from botocore<1.43
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in /usr/lib/python3.9/site-packages (from botocore
1)
Requirement already satisfied: six>=1.5 in /usr/lib/python3.9/site-packages (from python-dateutil<3.0.0,>=2.1
to3) (1.15.0)
Installing collected packages: botocore, s3transfer, boto3
Successfully installed boto3-1.42.65 botocore-1.42.65 s3transfer-0.16.0
[ec2-user@ip-172-31-3-119 ~]$ git --version
git version 2.50.1
[ec2-user@ip-172-31-3-119 ~]$ flask --version
Python 3.9.25
Flask 3.1.3
Werkzeug 3.1.6
[ec2-user@ip-172-31-3-119 ~]$
```

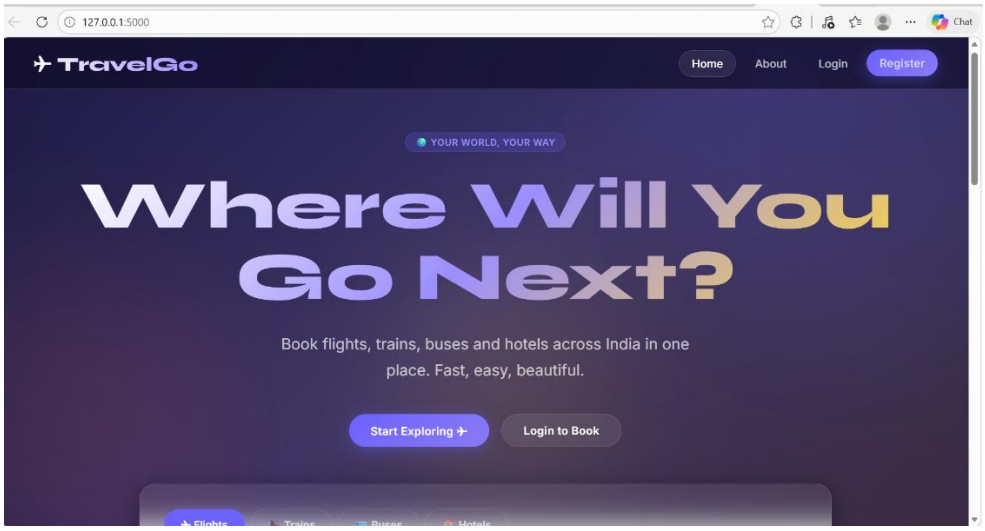
i-087d6d3aecbf03930 (travelgo)

PublicIPs: 98.80.201.173 PrivateIPs: 172.31.3.119

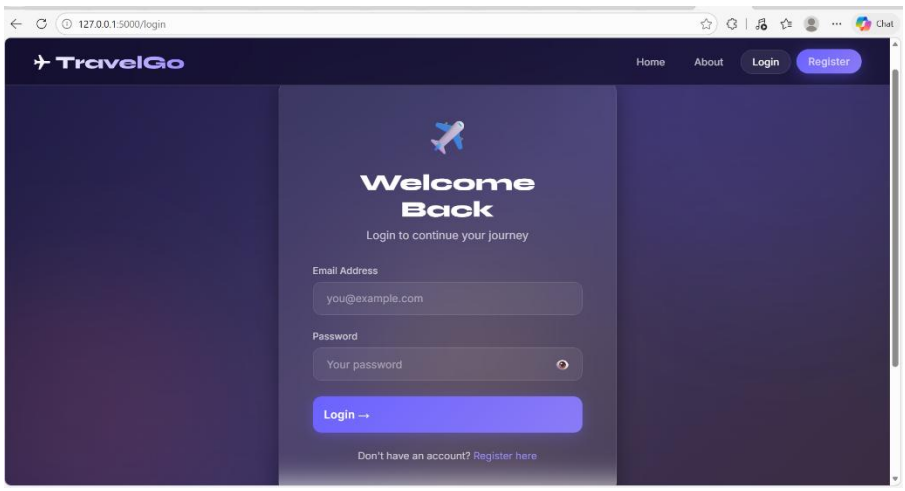
Launch the application:

```
ast login: Mon Apr 14 16:27:44 2025 from 13.233.177.5
buntu@ip-172-31-12-110:~$ cd Travel-Booking-website_Ec2_Dynamodb_SNS
buntu@ip-172-31-12-110:~/Travel-Booking-website_Ec2_Dynamodb_SNS$ cd travel_booking_system
buntu@ip-172-31-12-110:~/Travel-Booking-website_Ec2_Dynamodb_SNS/travel_booking_system$ export SNS_TOPIC_ARN=arn:aws:sns:ap-south-1:557690616836:Booki
buntu@ip-172-31-12-110:~/Travel-Booking-website_Ec2_Dynamodb_SNS/travel_booking_system$ git pull origin main
emote: Enumerating objects: 9, done.
emote: Counting objects: 100% (9/9), done.
emote: Compressing objects: 100% (4/4), done.
emote: Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
npacking objects: 100% (5/5), 1.68 KiB | 429.00 KiB/s, done.
rom https://github.com/AlekhyaPenubakula/Travel-Booking-website_Ec2_DynamodB_SNS
* branch            main       -> FETCH_HEAD
  2289f47..397e3a1  main       -> origin/main
pdating 2289f47..397e3a1
ast-forward
travel_booking_system/templates/hotels.html | 107 ++++++
```

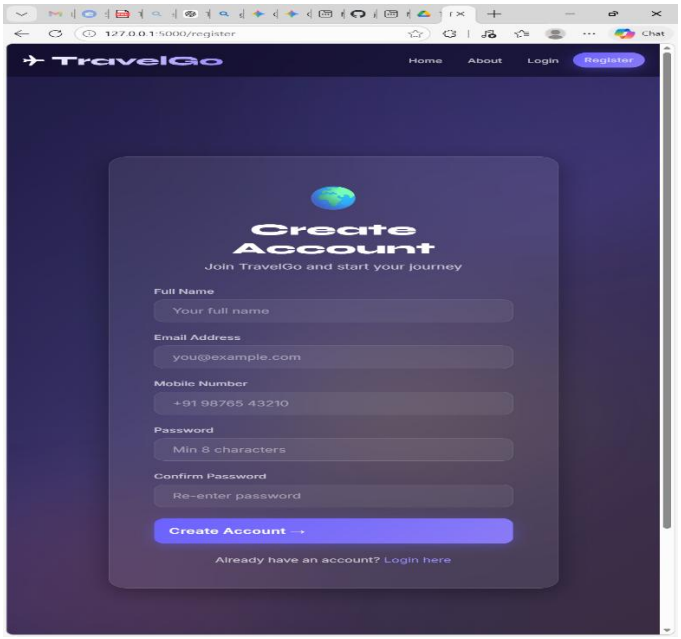
- **Home Page**



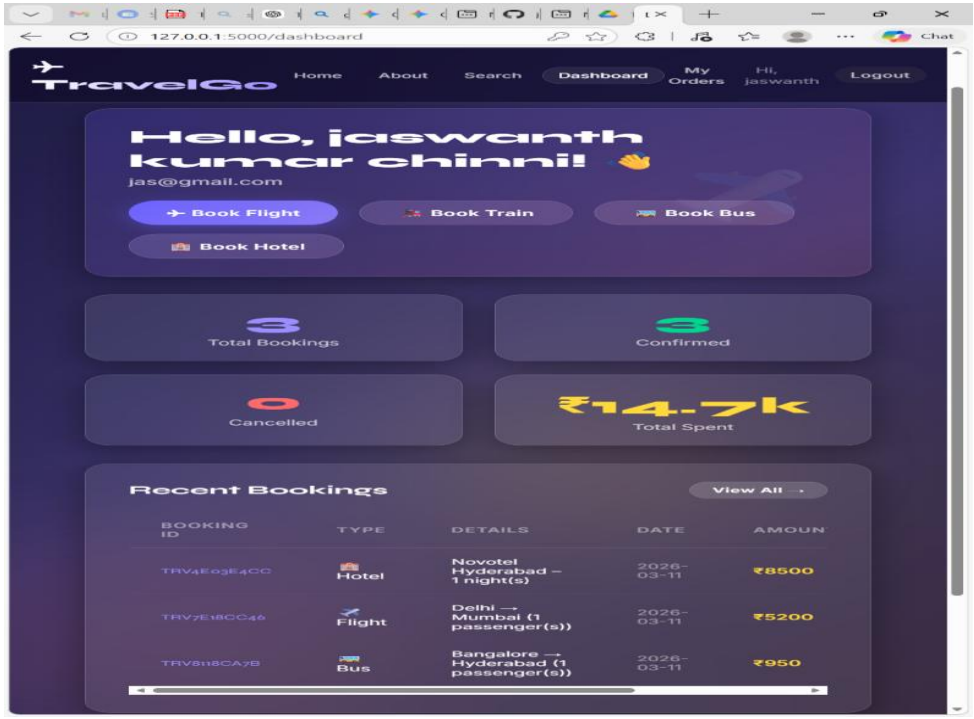
Login Page



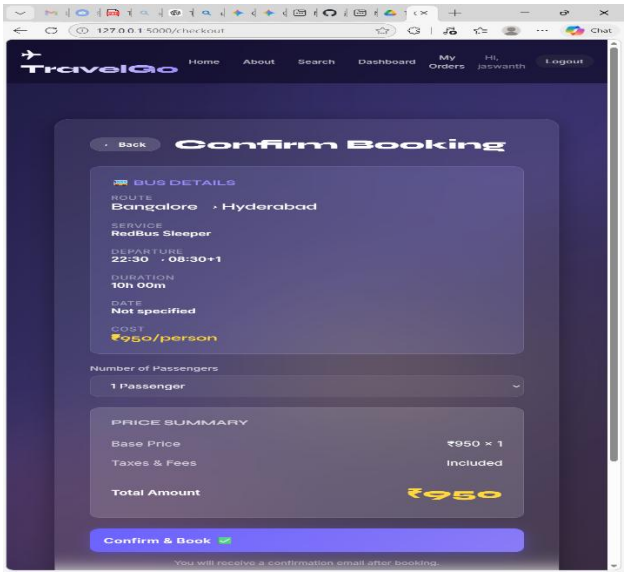
- **Register Page**

- 

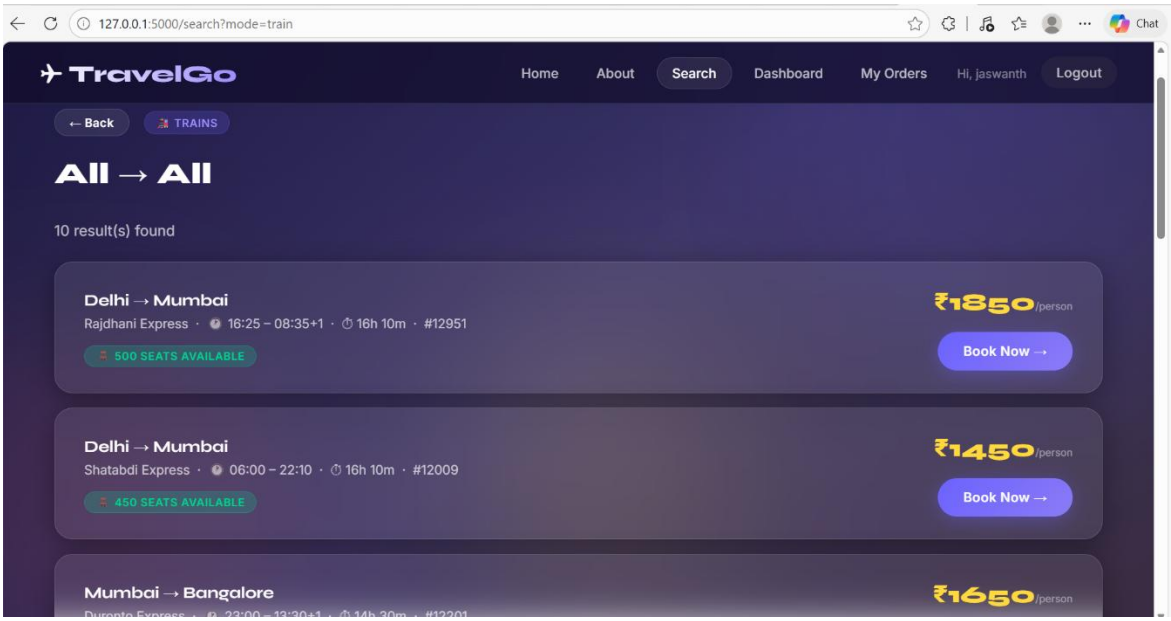
- **Dashboard Page**



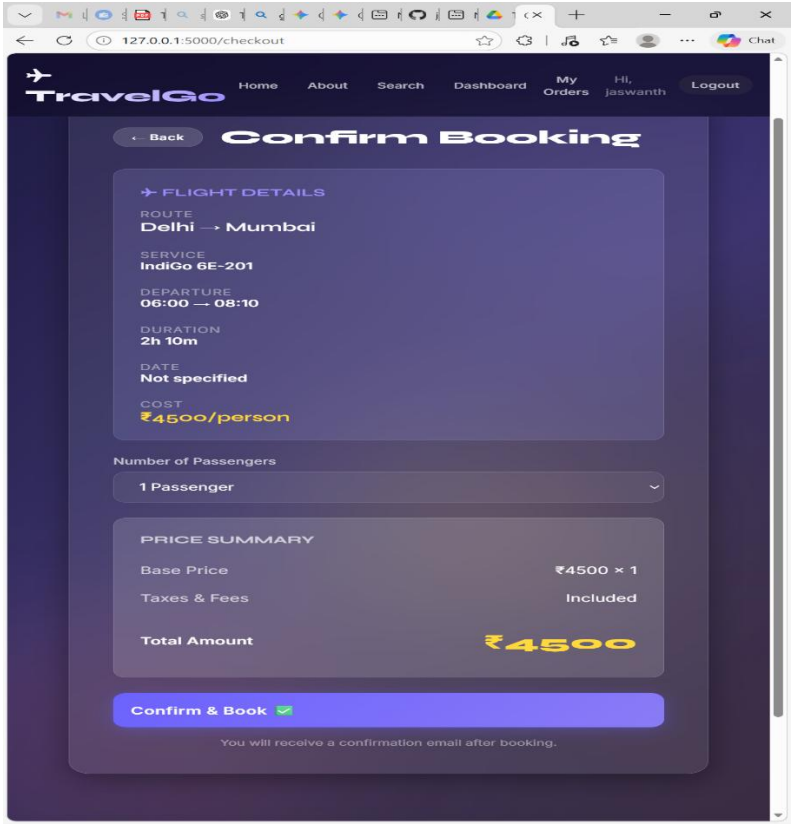
- **Buses Page**

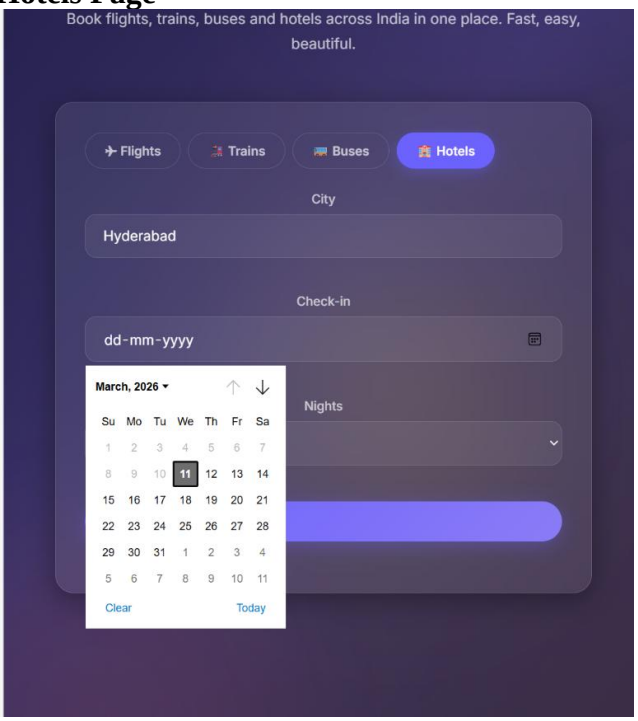


- **Trains Page**

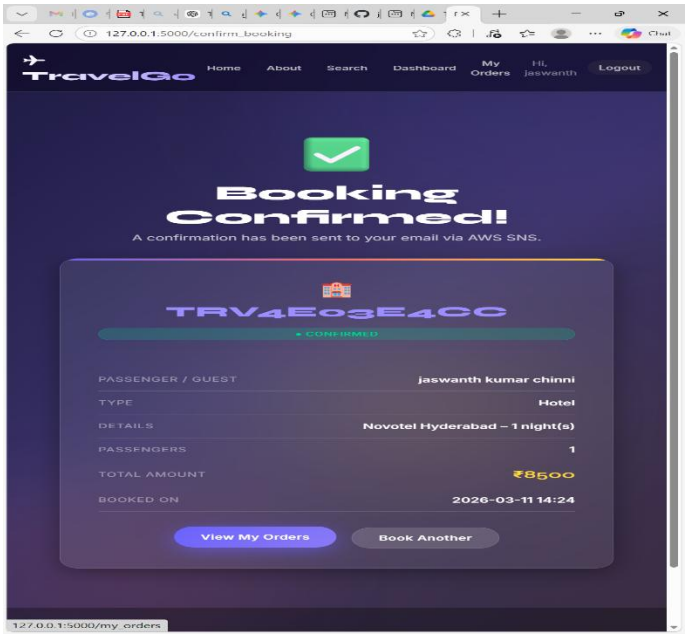


- **Flights Page**

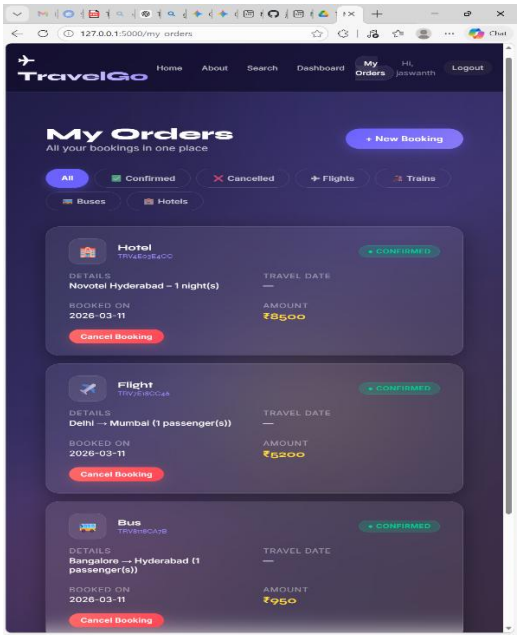

127.0.0.1:5000/checkout
TravelGo
Home
About
Search
Dashboard
My Orders
Hi, jaswanth
Logout
Back
Confirm Booking
FLIGHT DETAILS
ROUTE
Delhi → Mumbai
SERVICE
IndiGo 6E-201
DEPARTURE
06:00 → 08:10
DURATION
2h 10m
DATE
Not specified
COST
₹4500/person
Number of Passengers
1 Passenger
PRICE SUMMARY
Base Price
₹4500 × 1
Taxes & Fees
Included
Total Amount
₹4500
Confirm & Book
You will receive a confirmation email after booking.

- **Hotels Page**
- 
Book flights, trains, buses and hotels across India in one place. Fast, easy, beautiful.
Flights
Trains
Buses
Hotels
City
Hyderabad
Check-in
dd-mm-yyyy
March, 2026
Su Mo Tu We Th Fr Sa
Nights
Clear
Today

- **Payment Page**



- **Dashboard with Bookings**



DynamoDB Database Updates

1. **users Table**

Table: users - Items returned (4)

user_id (String)	name (String)	email (String)	mobile (String)	password (String)
u001	John Doe	johndoe@email.com	+11234567890	password123
u002	kilarukusuma	kilarukusuma12@gmail.com	+919876543210	123
u003	sunder	sunder@thesmartbridge.com	+1111111111	12345
u004	P_Alekhya	alekhya080228@gmail.com	+911122334455	1234

Store user credentials, login counts, and names in the **users** table for authentication and tracking.

2. **Transport table**

t_id (String)	type (String)	source (String)	destination (String)	journey_date (String)	journey_date (String)	cost_per_ticket (Number)
t001	Air	New York	London	2025-05-15	2025-05-15	500.00
t002	Bus	Chennai	Bangalore	2025-04-26	30.50	30.50
t003	Train	Paris	Rome	2025-06-10	2025-06-10	120.00
t004	Air	Singapore	Tokyo	2025-07-01	850.75	850.75

Store the details of all ways of transport

3.**hotels table**

hotel_id (String)	hotel_name (String)	hotel_location (String)	cost_per_room (Number)
h001	Grand Hyatt	New York	450.00
h002	Hotel Minerva	Chennai	150.00
h003	Paris Hilton	Paris	380.50
h004	Aparthotel	Rome	210.00

Store data of the hotels in their location with cost per room

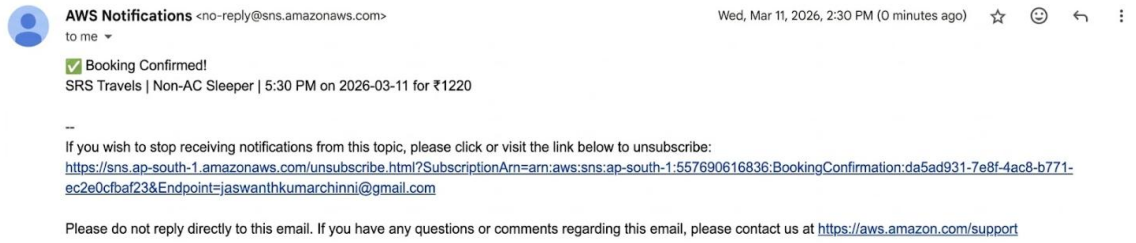
4. **bookings Table**

booking_id (String)	user_id (String)	t_id (String)	hotel_id (String)	book_date (String)	status (String)
7bc700fe	u002	t002	h002	2025-04-18	Confirmed
5af93005	u003	t003	h003	2025-04-18	Confirmed
6026a7a6	u004	t004	h004	2025-04-18	Confirmed
965e4155	u002	t001	h001	2025-04-18	Confirmed

Store travel bookings including **date, destination, transport details, and payment information**.

3. Mail to the User

Users receive **booking confirmation and cancellation notifications** through SNS email


AWS Notifications <no-reply@sns.amazonaws.com>
to me
Wed, Mar 11, 2026, 2:30 PM (0 minutes ago)

✅ Booking Confirmed!
SRS Travels | Non-AC Sleeper | 5:30 PM on 2026-03-11 for ₹1220

--
If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
https://sns.ap-south-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:ap-south-1:557690616836:BookingConfirmation:da5ad931-7e8f-4ac8-b771-ec2e0cfbaf23&Endpoint=jaswanthkumarchinni@gmail.com

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at https://aws.amazon.com/support

---

Conclusion

The **TravelGo Website** has been successfully developed and deployed using a **cloud-native AWS architecture**.

Using **EC2 for hosting**, **DynamoDB for real-time data storage**, and **SNS for notifications**, the system provides a seamless travel booking experience.

Users can search and book **buses, trains, flights, and hotels** through a centralized platform. The system supports efficient authentication, booking workflows, and real-time communication.

Overall, TravelGo demonstrates how **cloud technologies and web frameworks** can be combined to build scalable and user-friendly travel management platforms.